Universidad Nacional Autónoma de México

Facultad de Ingeniería

Laboratorio de Arquitectura y Organización de Computadoras

Grupo: 07 - Semestre: 2027-1

Reporte Práctica 6 pt 2.

Secuenciador básico de 4 microinstrucciones

Fecha de Entrega

16 de Octubre del 2026

Profesor

Ing. Julio Cesar Cruz Estrada

Integrantes:

Espinoza Matamoros Percival Ulises

Montiel Juárez Oscar Iván

Veraza García Amy Valentina

1. Objetivo

Familiarizar al alumno con el conocimiento del secuenciador básico, el cual es parte fundamental del procesador.

2. Desarrollo

Para esta práctica se partió del secuenciador básico alternativo implementado en la práctica anterior. En esta ocasión se modificó la arquitectura para que el recorrido de microinstrucciones se determine a partir de la ROM, en lugar de tener direcciones fijas.

El contador de microprograma (`mPC`) guarda la dirección presente. Esa dirección consulta la ROM, se separa en campos y alimenta la ruta de control. Donde el bloque de prueba selecciona una de las entradas X, Y, `INT` , mientras que el valor VF permite invertir la condición por medio de una compuerta XOR. Finalmente, el campo MI decide si la siguiente dirección corresponde al paso continuo, la liga, el registro de transformación o el registro de interrupción.

2.1. Carta ASM y tamaño de la memoria

La carta ASM que se implementó corresponde a la que se ve en la Figura[1],donde la rama izquierda corresponde a la prueba X, la rama central a la prueba Y y la rama derecha a la interrupción. Los rombos marcados en blanco representan los saltos de transformación, estos fueron implementados en el sistema como una señal de entrada de dos bits (T). Donde desde el estado E_1 con $T = 00, 11$ llega a E_2, $T = 01$ llega a E_8 y $T = 10$ llega a E_{13}, tanto desde E_{11}, $T = 01$ llega a E_7 y los demás valores llegan a E_{12}. La posición E_{15} se usa solamente como retorno de seguridad de la ROM, por lo que no forma parte de la carta ASM original.

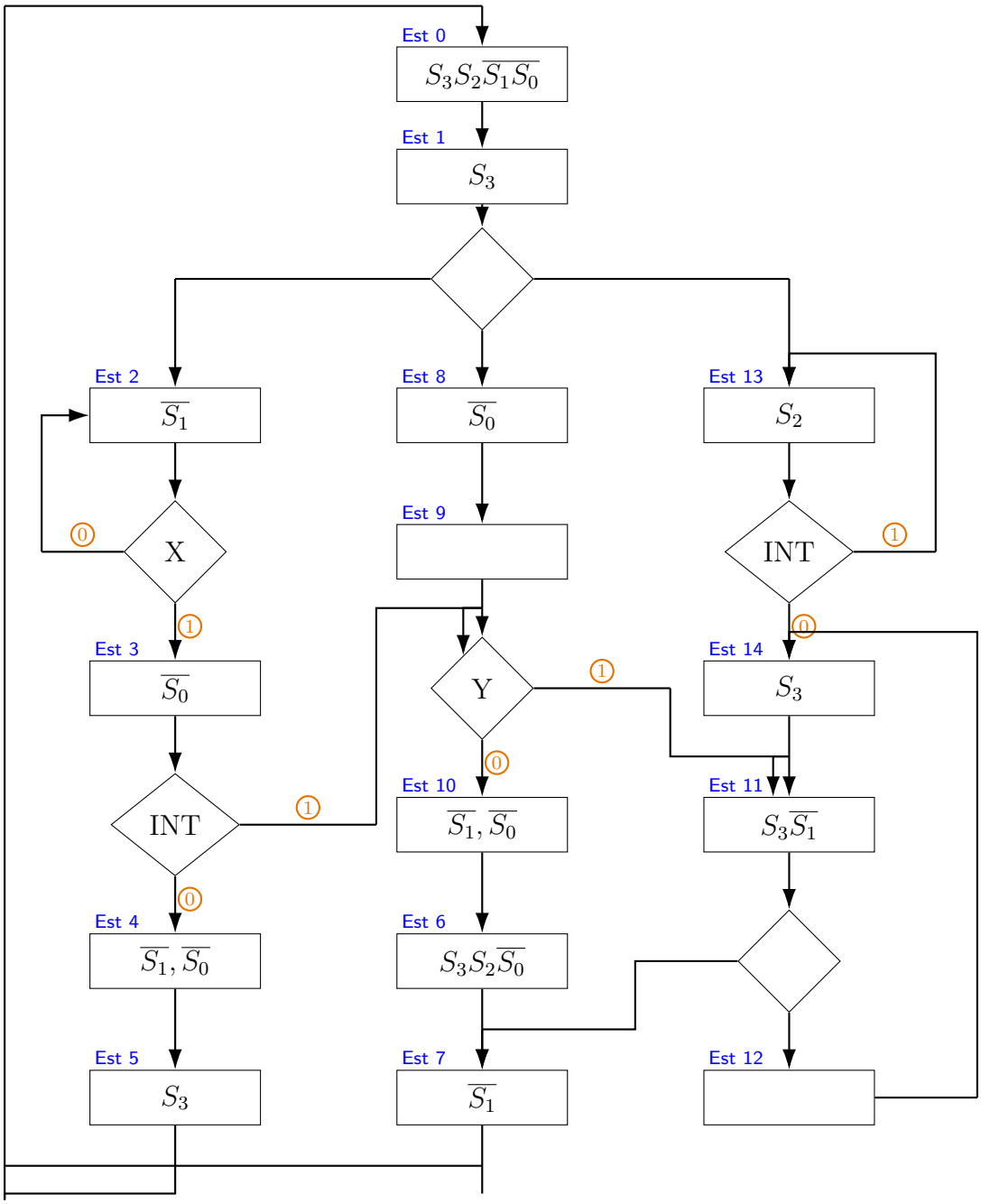


Figura 1: Carta ASM Práctica 6 - Parte 2.

La memoria requiere cuatro bits de dirección, ya que la microprogramación ocupa las posiciones de 0 a 15. Cada microinstrucción tiene un campo de prueba de dos bits, un bit de valor de prueba, dos bits para el tipo de microinstrucción, cuatro bits de liga y cuatro bits de salidas. Por lo tanto, su organización es de 16 palabras por 13 bits:

De forma que se requiere una memoria de control con capacidad de 208 bits:

$$n = 2^4 = 16 \text{ palabras}, \qquad m = 2 + 1 + 2 + 4 + 4 = 13 \text{ bits}$$

$$\text{Capacidad} = 16 \times 13 = \boxed{208 \text{ bits}}.$$

Tabla 1: Formato de una microinstrucción de la memoria de control.

Campo	Prueba	VF	MI	Liga	Salidas
Bits	K_1K_0	1	I_1I_0	$L_3L_2L_1L_0$	$S_3S_2S_1S_0$
Longitud	2	1	2	4	4

Los valores de MI codifican cuatro opciones: Con `00` selecciona $mPC + 1$, `01` selecciona la liga cuando la condición se cumple, `10` selecciona el registro de transformación y `11` selecciona el registro de interrupción. El bloque `muxEntradas` define la entrada que se prueba: `00` corresponde a X, `01` a Y, `10` a `INT` y `11` a una constante cero que no se emplea en todo el sistema.

2.2. Contenido de la memoria de control

En la Tabla [2] se tiene la microprogramación que fue implementada en el módulo `rom.vhd`. Los campos indiferentes de la carta ASM se guardaron con un valor definido, por lo que cada dirección de la ROM entrega una palabra completa de 13 bits.

Tabla 2: Contenido de la ROM para la carta ASM.

Dirección	Prueba	VF	MI	Liga	Salidas	Operación / siguiente estado
`0000`	`11`	0	`00`	`0000`	`1100`	Paso continuo a E_1.
`0001`	`11`	0	`10`	`0000`	`1011`	Transformación desde E_1.
`0010`	`00`	0	`01`	`0010`	`0001`	Prueba X: $0 \to E_2$, $1 \to E_3$.
`0011`	`10`	1	`11`	`0000`	`0000`	Salto por interrupción desde E_3.
`0100`	`11`	0	`00`	`0000`	`0000`	Paso continuo a E_5.
`0101`	`11`	0	`01`	`0000`	`1011`	Retorno incondicional a E_0.
`0110`	`11`	0	`00`	`0000`	`1110`	Paso continuo a E_7.
`0111`	`11`	0	`01`	`0000`	`0001`	Retorno incondicional a E_0.
`1000`	`11`	0	`00`	`0000`	`0010`	Paso continuo a E_9.
`1001`	`01`	1	`01`	`1011`	`0011`	Prueba Y: $1 \to E_{11}$, $0 \to E_{10}$.
`1010`	`11`	0	`01`	`0110`	`0000`	Salto incondicional a E_6.
`1011`	`11`	0	`10`	`0000`	`1001`	Transformación desde E_{11}.
`1100`	`11`	0	`01`	`1110`	`0011`	Salto incondicional a E_{14}.
`1101`	`10`	1	`11`	`0000`	`0111`	Salto por interrupción desde E_{13}.
`1110`	`11`	0	`01`	`1011`	`1011`	Salto incondicional a E_{11}.
`1111`	`11`	0	`01`	`0000`	`0011`	Posición de seguridad: regreso a E_0.

2.3. Arquitectura implementada

Como se mencionó anteriormente para la implementación del sistema se partió del secuenciador básico alternativo que se implementó en la practica anterior. Sin embargo se le hicieron modificaciones, la principal diferencia es que por motivos de conexión, los bloques relacionados en el registro de transformación e interrupción se implementaron como bloques fuera del bloque correspondiente al `secuenciador`.

El sistema implementado consta del siguiente diagrama de bloques, para la señal de reloj al igual que en prácticas anteriores se utilizó un generador de paso por botón, el cual secuenciador alternativo, este recibe varios datos de entrada: registro de transformación, registro de interrupción, liga, microinstrucción, etc. La mayoría de estos datos de entrada soon generados más adelante en el sistema, el secuenciador tiene como salida el `mpc` y el `estado presente`, el `mpc` se retroalimenta al mismo secuenciador, mientras que el `estado presente` se conecta a la memoria ROM, la cual genera la microinstrucción, la cual se separa en varios campos, (salidas, liga, prueba, VF, MI) los cuales van conectados a los bloques correspondientes (secuenciador y mux entradas). Por otro lado el estado presente también se conecta a los registros de transformación e interrupción, los cuales generan lo9s registros necesarios para conectarlos al secuenciador.

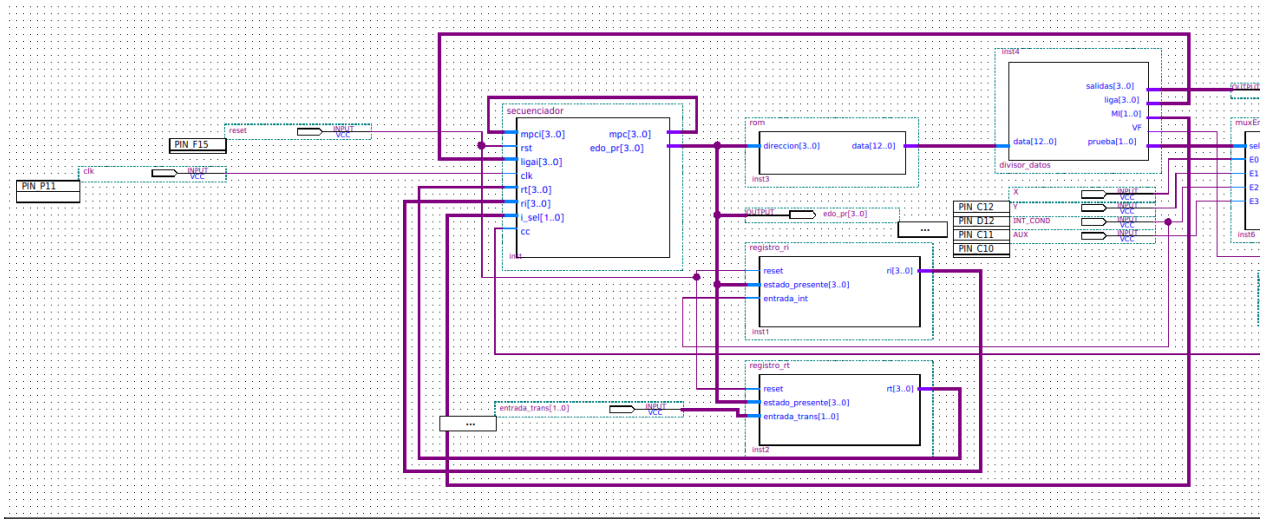


Figura 2: Interconexión de la ruta de microdirección del secuenciador.

Por ultimo se tienen dos bloques `divisor_datos` y `mux_entradas`, el primero se encarga de separar los campos de la microinstrucción, mientras que el segundo selecciona la condición que se va a evaluar, ya sea X, Y o `INT`. Adicionalmente se tiene una compuerta XOR entre el VF y la salida del `mux_entradas`, la cual genera la condición final que se va a evaluar en el secuenciador.

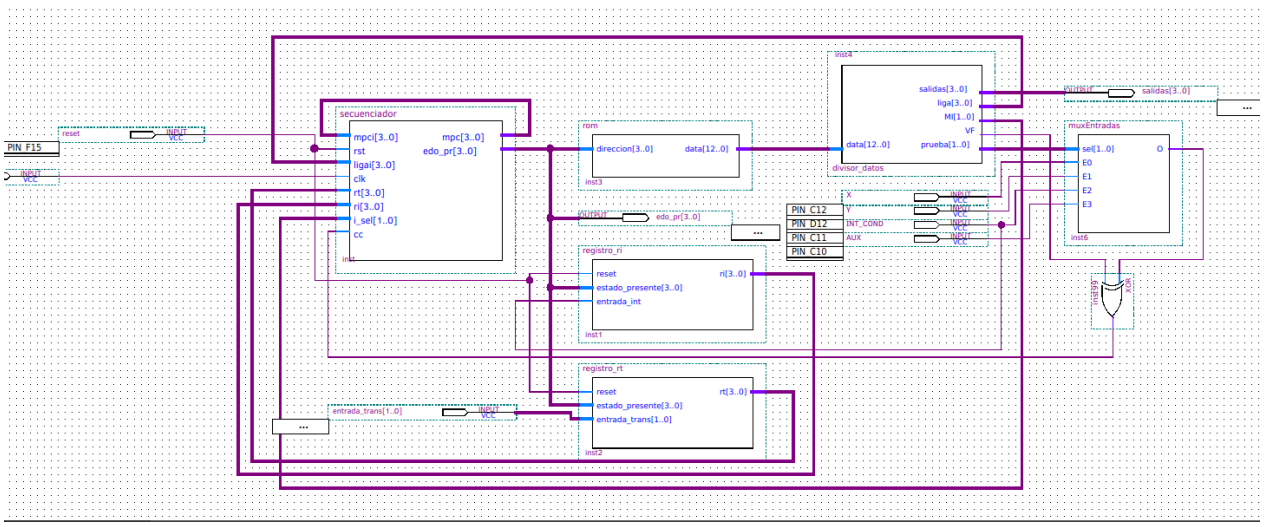


Figura 3: Interconexión de la memoria ROM, el divisor de datos y la ruta de control.

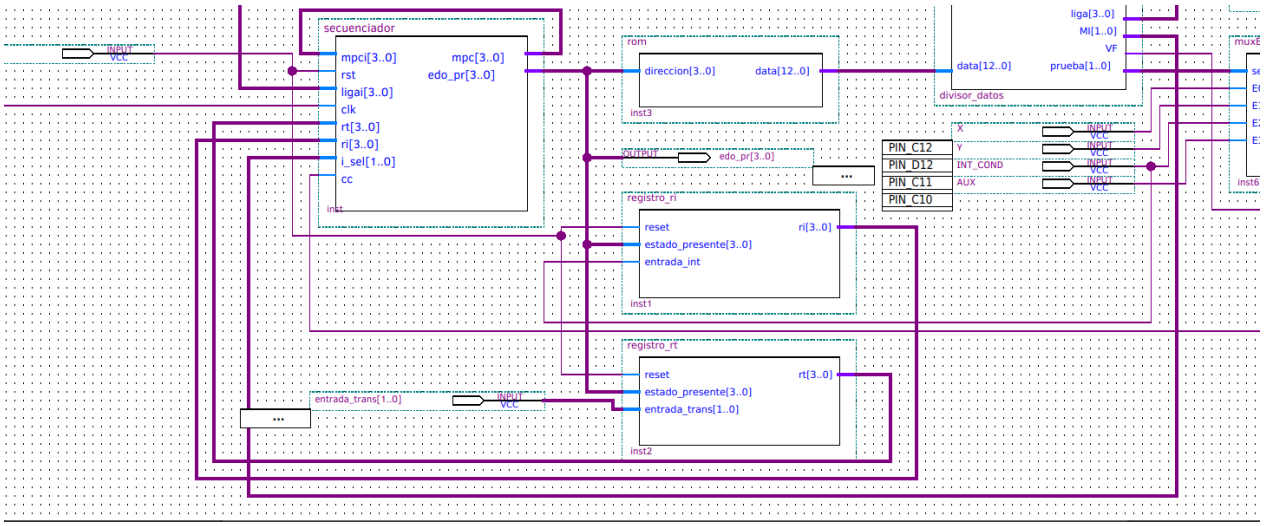


Figura 4: Diagrama final de bloques del secuenciador microprogramado.

2.4. Módulos implementados en VHDL

Memoria y divisor de campos : La memoria ROM y el divisor de datos son componentes clave del sistema, `rom` almacena las dieciséis microinstrucciones de 13 bits. `divisor_datos` separa cada palabra en los campos de prueba, inversión, modo, liga y salidas.

Listing 1: Memoria ROM de 16 palabras por 13 bits.

```
1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3 use IEEE.STD_LOGIC_ARITH.ALL;
```

```
4  use IEEE.STD_LOGIC_UNSIGNED.ALL;
5
6  entity rom is
7      Port (
8          direccion : in  std_logic_vector(3 downto 0);
9          data      : out std_logic_vector(12 downto 0)
10     );
11 end rom;
12
13 architecture Behavioral of rom is
14
15     type mem is array (0 to 15) of std_logic_vector(12 downto 0);
16
17     -- Formato de la microinstruccion (13 bits):
18     -- data(12 downto 11) = Prueba K1,K0
19     -- data(10)           = VF
20     -- data(9 downto 8)   = MI M1,M0
21     -- data(7 downto 4)   = Liga L3,L2,L1,L0
22     -- data(3 downto 0)   = Salidas S3,S2,S1,S0
23     --
24     -- MI: 00 = avance (MPC+1), 01 = salto condicional por Liga,
25     --     10 = salto a RT,     11 = salto por interrupcion a RI.
26     -- Los '*' de la tabla del reporte se sustituyen por ceros o por
27     -- una prueba sin uso, ya que son campos indiferentes en esa fila.
28     constant ROM : mem := (
29         -- Dir  Prueba VF MI  Liga  Salidas
30         0  => "11" & "0" & "00" & "0000" & "1100", -- Carta ASM: S3 S2 /S1 /S0
31         1  => "11" & "0" & "10" & "0000" & "1011",
32         2  => "00" & "0" & "01" & "0010" & "0001",
33         3  => "10" & "1" & "11" & "0000" & "0000",
34         4  => "11" & "0" & "00" & "0000" & "0000",
35         5  => "11" & "0" & "01" & "0000" & "1011",
36         6  => "11" & "0" & "00" & "0000" & "1110",
37         7  => "11" & "0" & "01" & "0000" & "0001",
38         8  => "11" & "0" & "00" & "0000" & "0010",
39         9  => "01" & "1" & "01" & "1011" & "0011",
40         10 => "11" & "0" & "01" & "0110" & "0000",
41         11 => "11" & "0" & "10" & "0000" & "1001",
42         12 => "11" & "0" & "01" & "1110" & "0011",
43         13 => "10" & "1" & "11" & "0000" & "0111",
44         14 => "11" & "0" & "01" & "1011" & "1011",
45         15 => "11" & "0" & "01" & "0000" & "0011"
46     );
47
48 begin
49
50     process(direccion)
51     begin
52         data <= ROM(conv_integer(unsigned(direccion)));
53     end process;
54
55 end Behavioral;
```

Listing 2: Divisor de los campos de una microinstrucción.

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3
4  entity divisor_datos is
5      Port (
6          data    : in  std_logic_vector(12 downto 0);
7          prueba  : out std_logic_vector(1 downto 0);
8          VF      : out std_logic;
```

```
        MI      : out std_logic_vector(1 downto 0);
        liga    : out std_logic_vector(3 downto 0);
        salidas : out std_logic_vector(3 downto 0)
    );
end divisor_datos;

architecture Behavioral of divisor_datos is
begin
    -- Prueba[12:11], VF[10], MI[9:8], Liga[7:4], Salidas[3:0]
    prueba  <= data(12 downto 11);
    VF      <= data(10);
    MI      <= data(9 downto 8);
    liga    <= data(7 downto 4);
    salidas <= data(3 downto 0);
end Behavioral;
```

Selección de condiciones y direcciones. : `muxEntradas` selecciona una de las entradas X, Y, `INT` o `AUX`. `logica` interpreta el campo MI y determina si se avanza, se usa la liga o se salta a transformación o interrupción. `incrementador` calcula la siguiente dirección secuencial y `mux` selecciona entre ella, la liga, `RT` o `RI`.

Es importante destacar que para implementar la lógica del registro de interrupción y transformación, se decidió considerarlo como señales de entrada para poder seleccionar que registro se va a utilizar, por otro lado la lógica implementada era por medio de una sentencia condicional evaluar la señal de entrada correspondiente a cada registro, de forma que se evalúa si el estado presente es igual a los estados donde hay un salto a transformación o interrupción, de ser así se selecciona la señal de entrada correspondiente, de lo contrario continua el recorrido normal de microinstrucciones.

Listing 3: Multiplexor de entradas para la condición de control.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity muxEntradas is
    Port (
        sel : in  std_logic_vector(1 downto 0);
        E0  : in  std_logic; -- X
        E1  : in  std_logic; -- Y
        E2  : in  std_logic; -- INT
        E3  : in  std_logic; -- AUX
        O   : out std_logic
    );
end muxEntradas;

architecture Behavioral of muxEntradas is
begin
    process(sel, E0, E1, E2, E3)
    begin
        O <= '0';
        case sel is
            when "00"   => O <= E0;
            when "01"   => O <= E1;
            when "10"   => O <= E2;
            when "11"   => O <= E3;
            when others => O <= '0';
        end case;
```

```
    end process;
end Behavioral;
```

Listing 4: Lógica que decodifica el tipo de microinstrucción.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity logica is
    Port (
        i_sel : in  std_logic_vector(1 downto 0); -- i_sel(1)=i_1, i_sel(0)=i_0
        cc    : in  std_logic;                    -- Condición de control/prueba
        sel   : out std_logic_vector(1 downto 0)  -- Selector de modo
    );
end logica;

architecture Behavioral of logica is
begin
    process(i_sel, cc)
    begin
        sel <= "00";
        case i_sel is
            when "00" =>
                sel <= "00";
            when "01" =>
                if cc = '0' then
                    sel <= "01"; -- Salto condicional a Liga de la ROM
                else
                    sel <= "00"; -- Paso continuo
                end if;
            when "10" =>
                sel <= "10"; -- Salto a transformacion
            when "11" =>
                sel <= "11"; -- RI decide la direccion con estado e INT
            when others =>
                sel <= "00";
        end case;
    end process;
end Behavioral;
```

Listing 5: Incrementador de la microdirección.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity incrementador is
    Port ( edop: in std_logic_vector (3 downto 0);
           edouno : out std_logic_vector (3 downto 0)
         );
end incrementador;

architecture Behavioral of incrementador is
begin

    process(edop)
    begin

        edouno <= edop + 1;

    end process;
```

```
21
22 end Behavioral;
```

Listing 6: Multiplexor de la siguiente microdirección.

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.STD_LOGIC_ARITH.ALL;
4  use IEEE.STD_LOGIC_UNSIGNED.ALL;
5
6  entity mux is
7      Port ( mpc : in std_logic_vector (3 downto 0);
8             liga : in std_logic_vector (3 downto 0);
9             rt : in std_logic_vector (3 downto 0);
10            ri : in std_logic_vector (3 downto 0);
11            sel: in std_logic_vector (1 downto 0);
12            edop: out std_logic_vector (3 downto 0)
13          );
14 end mux;
15
16 architecture Behavioral of mux is
17 begin
18 process(mpc, liga, rt, ri, sel)
19     begin
20         if sel = "00" then
21             edop <= mpc;  -- Paso continuo (mpc + 1)
22         elsif sel = "01" then
23             edop <= liga; -- Salto condicional a Liga de la ROM
24         elsif sel = "10" then
25             edop <= rt;   -- Salto a transformación (RT)
26         elsif sel = "11" then
27             edop <= ri;   -- Salto a interrupción (RI)
28         end if;
29     end process;
30 end Behavioral;
```

Registros y avance por botón. `registro_mpc` conserva la microdirección y la actualiza en cada flanco ascendente, con reinicio a cero. `registro_rt` y `registro_ri` generan las direcciones de transformación e interrupción según el estado y sus entradas. `clk_buttom` es un divisor de frecuencia que genera un pulso de avance por cada pulsación del botón.

Listing 7: Registro del contador de microprograma.

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3
4  entity registro_mpc is
5      Port (
6          reloj     : in  std_logic;
7          reset     : in  std_logic; -- KEY1, activo en 0
8          entrada_r : in  std_logic_vector(3 downto 0);
9          mpc       : out std_logic_vector(3 downto 0)
10     );
11 end registro_mpc;
12
13 architecture Behavioral of registro_mpc is
14     signal estado : std_logic_vector(3 downto 0) := "0000";
15 begin
16     process(reloj, reset)
```

```
    begin
        if reset = '0' then
            estado <= "0000";
        elsif rising_edge(reloj) then
            estado <= entrada_r;
        end if;
    end process;

    mpc <= estado;
end Behavioral;
```

Listing 8: Registro de transformación.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity registro_rt is
    Port (
        reset            : in  std_logic;
        estado_presente  : in  std_logic_vector(3 downto 0);
        entrada_trans    : in  std_logic_vector(1 downto 0);
        rt               : out std_logic_vector(3 downto 0)
    );
end registro_rt;

architecture Behavioral of registro_rt is
begin
    process(reset, estado_presente, entrada_trans)
    begin
        if reset = '0' then
            rt <= "0000";
        else
            case estado_presente is
                when "0001" => -- Estado 1
                    case entrada_trans is
                        when "00"   => rt <= "0010"; -- Estado 2
                        when "01"   => rt <= "1000"; -- Estado 8
                        when "10"   => rt <= "1101"; -- Estado 13
                        when others => rt <= "0010";
                    end case;

                when "1011" => -- Estado 11
                    case entrada_trans is
                        when "00"   => rt <= "1100"; -- Estado 12
                        when "01"   => rt <= "0111"; -- Estado 7
                        when others => rt <= "1100";
                    end case;

                when others =>
                    rt <= "0000";
            end case;
        end if;
    end process;
end Behavioral;
```

Listing 9: Registro de interrupción.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity registro_ri is
    Port (
```

```
        reset            : in  std_logic;
        estado_presente  : in  std_logic_vector(3 downto 0);
        entrada_int      : in  std_logic;
        ri               : out std_logic_vector(3 downto 0)
    );
end registro_ri;

architecture Behavioral of registro_ri is
begin
    process(reset, estado_presente, entrada_int)
    begin
        if reset = '0' then
            ri <= "0000";
        else
            case estado_presente is
                when "0011" => -- Estado 3: INT=0 -> 4; INT=1 -> 10
                    if entrada_int = '0' then
                        ri <= "0100";
                    else
                        ri <= "1010";
                    end if;

                when "1101" => -- Estado 13: INT=0 -> 14; INT=1 -> 0
                    if entrada_int = '0' then
                        ri <= "1110";
                    else
                        ri <= "0001";
                    end if;

                when others =>
                    ri <= "0000";
            end case;
        end if;
    end process;
end Behavioral;
```

Listing 10: Generador de un avance por pulsación.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity clk_buttom is
    port(
        clk : in std_logic;
        buttom : in std_logic;
        div_clk : out std_logic
    );
end clk_buttom;

architecture Behavioral of clk_buttom is

CONSTANT STATE_START: std_logic_vector(1 downto 0) := "00";
CONSTANT STATE_COUNT: std_logic_vector(1 downto 0) := "01";
CONSTANT STATE_DONE: std_logic_vector(1 downto 0) := "10";

CONSTANT MAX_CLK_COUNT: std_logic_vector(27 downto 0) := X"17D7840"; -- 25 millones de
    pulsos

signal state: std_logic_vector(1 downto 0) := "00";
signal cuenta: std_logic_vector(27 downto 0) := X"0000000";

```

```
25  begin
26      process(clk, buttom)
27      begin
28          if rising_edge(clk) then
29              case state is
30                  WHEN STATE_START =>
31                      if buttom = '1' then
32                          state <= STATE_COUNT;
33                      end if;
34                  WHEN STATE_COUNT =>
35                      if cuenta >= MAX_CLK_COUNT then
36                          state <= STATE_DONE;
37                      end if;
38                  WHEN STATE_DONE =>
39                      state <= STATE_START;
40                  WHEN others =>
41                      state <= STATE_START;
42              end case;
43          end if;
44      end process;
45
46      process(clk)
47      begin
48          if rising_edge(clk) then
49              if state = STATE_COUNT then
50                  cuenta <= cuenta + 1;
51              else
52                  cuenta <= X"0000000";
53              end if;
54          end if;
55      end process;
56
57      process(state)
58      begin
59          if state = STATE_COUNT then
60              div_clk <= '1';
61          else
62              div_clk <= '0';
63          end if;
64      end process;
65          end Behavioral;
```

3. Simulación

La simulación se realizó aplicando una señal de reloj periódica, manteniendo `reset=1`, se fueron modificando las entradas X, Y, `INT`, `AUX` y `entrada_trans`. En las formas de onda se observan la dirección presente `edo_pr`, las salidas de la ROM `salidas`, el selector de prueba, las entradas de la carta ASM, el reloj y el reinicio. La dirección sólo cambia en los flancos ascendentes del reloj, por lo que `edo_pr` permanece estable durante el intervalo correspondiente a una microinstrucción.

En la Figura[5] se puede observar el recorrido inicial del secuenciador. La dirección parte de `0000`, donde la ROM entrega las salidas `1100` y el campo $MI =$ `00` selecciona el avance secuencial hacia `0001`. En esa dirección se activa el salto de transformación ($MI =$ `10`), por lo que la siguiente dirección depende de `entrada_trans`. En la forma de onda se pueden

apreciar varios recorridos posibles, como `0000` → `0001` → `0010` y `0000` → `0001` → `1000`. Estos recorridos corresponden a diferentes valores de la entrada de transformación: desde E_1 se puede dirigir el secuenciador a E_2, E_8 o E_{13}, dependiendo de la señal de entrada. Posteriormente, la microinstrucción de E_2 utiliza la prueba X y selecciona la liga o el avance según el resultado de la condición.

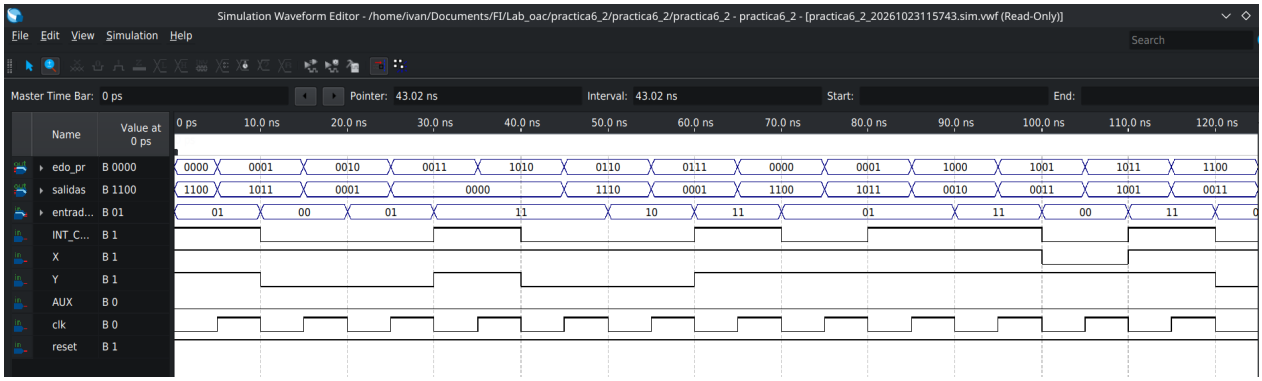


Figura 5: Recorrido inicial de las microdirecciones, salidas de la ROM y entradas de control.

La Figura [6] muestra una secuencia que inicia en `0011`, correspondiente a E_3. En este estado la microinstrucción utiliza MI = `11`, de modo que la dirección se obtiene del registro `RI` y depende de `INT`. Cuando `INT=0`, el registro `RI` produce `0100`, y el recorrido continúa por `0101` hasta regresar a `0000`. También se observa el recorrido `0001` → `1000` → `1001`, donde la primera transición corresponde a una transformación y, en E_9, el campo `Prueba` selecciona Y. Según el valor de Y, el secuenciador avanza a E_{10} o carga la liga de la microinstrucción.

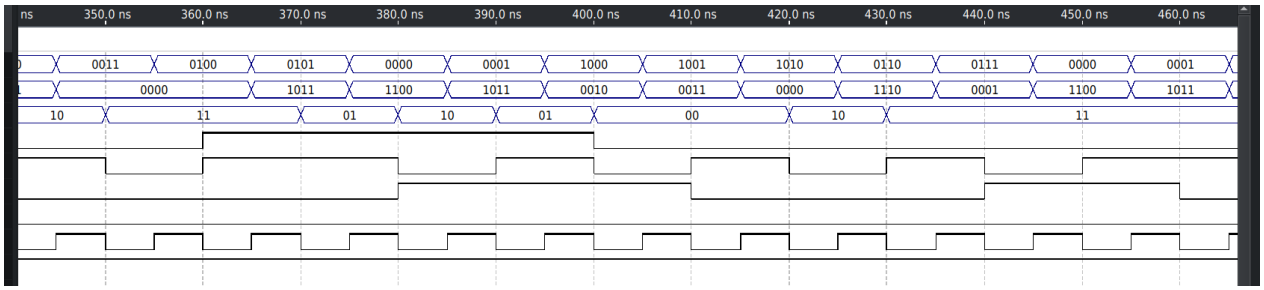


Figura 6: Recorridos con prueba de Y y salto por interrupción desde E_3.

En la Figura [7] se analiza la otra ruta de transformación e interrupción. La cual inicia desde `1011` (E_{11}), el registr de transformación `RT` conduce la secuencia a E_{12} o a E_7 según `entrada_trans`. Cuando se selecciona E_{12}, la liga `1110` lleva a E_{14} y la liga de E_{14} devuelve el control a E_{11}. En esta misma parte se observa el salto por interrupción desde E_{13}, cuando `INT=0` conduce a `1110`, mientras que `INT=1` conduce a `0001`, de acuerdo con la tabla de

microprogramación. Las señales `salidas` adoptan los valores almacenados en la ROM para cada dirección.

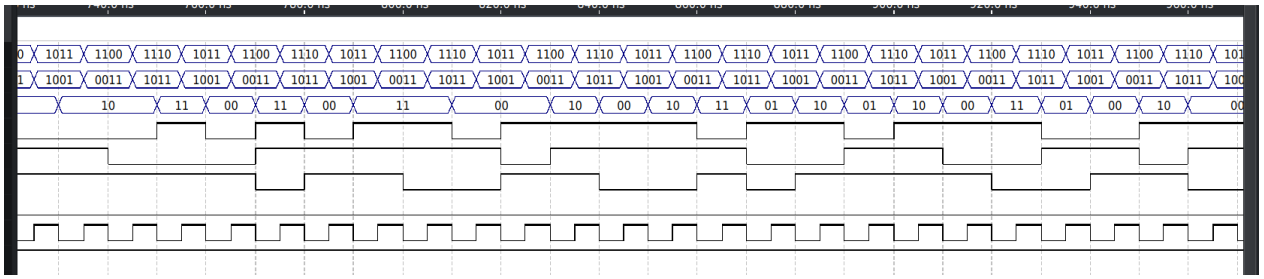
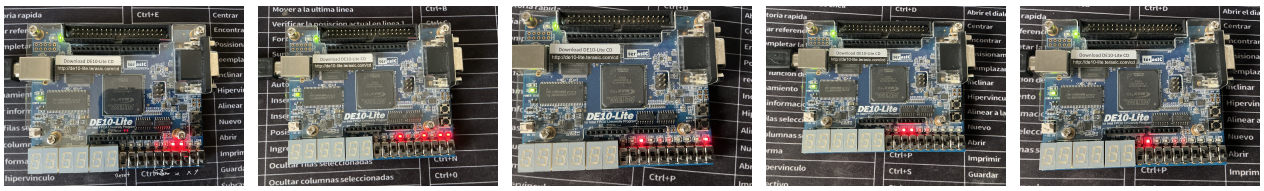


Figura 7: Recorridos de transformación e interrupción en la rama de E_{11} a E_{14}.

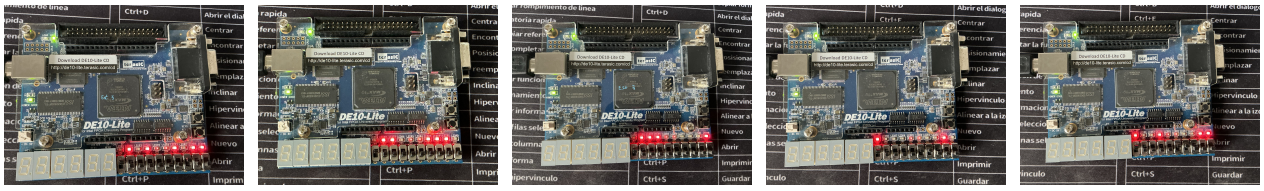
4. Resultados en la tarjeta FPGA

Al implementar el circuito en la FPGA DE10-Lite, se observan el estado presente mediante los LEDs. Los LEDs de la izquierda muestran las salidas de la carta ASM, mientras que los LEDs de la derecha indican el estado presente, a su vez cada que se presiona el botón, el secuenciador avanza un estado, permitiendo verificar la correspondencia entre la carta ASM y la implementación física.



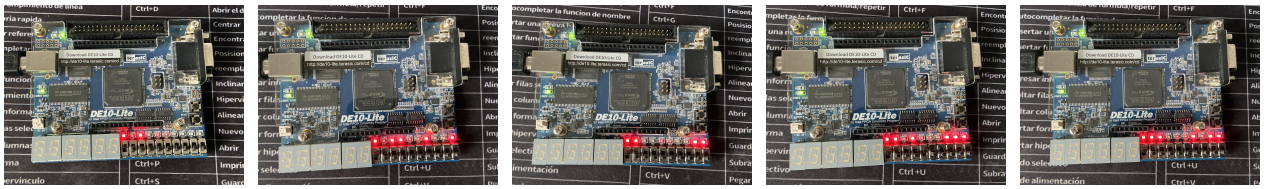
(a) E_0 (b) E_1 (c) E_2 (d) E_3 (e) E_4

Figura 8: Estados iniciales observados en la tarjeta DE10-Lite.



(a) E_5 (b) E_6 (c) E_7 (d) E_8 (e) E_9

Figura 9: Comprobación física de los estados E_5 a E_9.



(a) E_{10} (b) E_{11} (c) E_{12} (d) E_{13} (e) E_{14}

Figura 10: Comprobación física de los estados E_{10} a E_{14}.

5. Conclusión

Percival Ulises Espinoza Matamoros. El desarrollo de esta práctic me permitió comprender mejor la importancia del secuenciador. Al implementar la carta ASM con interrupciones y transformaciones, se observó que el secuenciador es un componente fundamental para el control del sistema, ya que genera la secuencia de microinstrucciones y selecciona la siguiente microdirección de acuerdo con las condiciones y el estado presente. A demás la implementación propuesta ara la transformación e interrupción permite poder implementar de manera sencilla y clara la carta ASM, ya que se puede cambiar el estado de transformación o interrupción sin necesidad de modificar el secuenciador.

Oscar Iván Montiel Juárez. En esta práctica comprendimos de manera integral cómo el secuenciador coordina el flujo de control de un procesador. Al construir el secuenciador básico alternativo de cuatro tipos de microinstrucción, asignamos a la ROM las direcciones y señales de control, y conectamos el contador de microprograma, los multiplexores y los registros de transformación e interrupción para determinar el estado siguiente. Esto nos permitió seguir las distintas rutas de la carta ASM y comprobar, tanto en la simulación como en la tarjeta, cómo las condiciones de entrada, las ligas y los saltos dirigen la ejecución.

Amy Valentina Veraza García. Esta práctica me ayudó a comprender cómo una carta ASM se convierte en una secuencia de microinstrucciones almacenadas en la ROM. Al separar los campos de prueba, inversión, modo, liga y salidas, pude identificar la función de cada bloque y entender por qué una misma dirección puede conducir a rutas distintas según las entradas. La simulación permitió revisar los saltos por transformación e interrupción, mientras que el avance por botón y los LEDs de la DE10-Lite hicieron posible verificar los estados y sus salidas en el circuito físico.